

Szakdolgozat

NEPTUN: XPGMCH

2025. december 2.

Tartalomjegyzék

1. Bevezetés	2
2. Feladat	3
2.1. Tervezés	4
2.2. Tervezett node-ok	5
3. Fejlesztés	6
3.1. Fejlesztő környezet beüzemelése	6
3.2. Szimuláció	6
3.2.1. Program	6
3.2.2. Szimulációs környezet	7
3.3. Kézi vezérlés	9
3.4. Kamerakép elemzése	10
3.4.1. Hough körök	10
3.4.2. Kontúr alapú körkeresés	13
3.5. Autómatizált kör keresése	18
3.5.1. A PID logika	18
3.5.2. Megvalósítás	18
4. Működése	23
4.1. A működés folyamata	23
4.2. A felhasználó által látható elemek	24
4.3. Tapasztalt problémák és hibajelenségek	24
4.4. Működés körülmények	25
5. Fejlesztési tervek	26
5.1. Jövőbeli bővítési pontok	26
6. Összegzés	27

1. Bevezetés

Miután gyerekoromban olyan filmeket mutattak nekem a szüleim, mint a Mátrix (r.: Wachowski testvérek) és a Terminátor második felvonása (r.: James Cameron). Ezekből a filmekből két dolog érdekelt leginkább: a robotok és a mesterséges intelligencia.

Ugyan középiskolai tanulmányait még úgy kezdtem meg, hogy kémiával fogok foglalkozni, de aztán az ott rengetek gyárat látogattunk meg (Tiszújvárosi MOL finomító, vagy magát a Borsodcham poliészter üzemét), így történt, hogy ott tudtam találkozni robotokkal és az automatizálás gyakorlati megvalósulásával. Közben pedig az iskolában elkezdtünk programozással foglalkozni, ami egy idő után jobban érdekelt, mint az egyébkénti fakultációs tantárgyam. Így kerültem végül a Miskolci Egyetem Gépészmérnöki és Informatika Karára, ahol én az utóbbiakat erősítem.

Közben kitört a háború Ukrajna és Oroszország között, így rengeteget lehetett arról olvasni, hogy a drónok mennyire megreformálták a hadviselést, ami szintén egy olyan téma, ami érdekelt. További érdekesség volt az automatizált, jobban mondva a robotizált légvédelem megjelenése, ami a földlevegő rakéták ellen volt jó. Ezekről a technológiákról szívesen olvastam.

Már első félévben kapcsolatba kerültem az egyetem robotikával foglalkozó csoportjával, de ekkor még főleg szabadidő hiány miatt nem tudtam foglalkozni a témával. Aztán az történt, hogy felvettem egy tantárgyat, aminek az volt. a neve, hogy "programozható logikák". Őszintén szólva csak kellett kredit, és a neve is érdekesen hangzott, de mint utólag kiderült robotokat működtető processzorokról volt szó, és a gyakorlati jegy is egy robotot kellett mozgásra bírni - aranytojáásra bukkantam ekkor.

Mivel nagyon élveztem a robot programozását, ezért a gyakorlatot tartó tanár - Bartók Roland - felajánlotta, hogy ha már következő féléveben fogok úgy is szakdolgozatot írni, akkor ő szívesen vállal engem, ha valami robot programozást szeretnék témának választani.

Így lett végül az a téma, hogy egy drónra írok programot, ami színes karikákat keres, majd a színüket felismerve, és egy előre megadott sorrend szerint átrepül rajtuk.

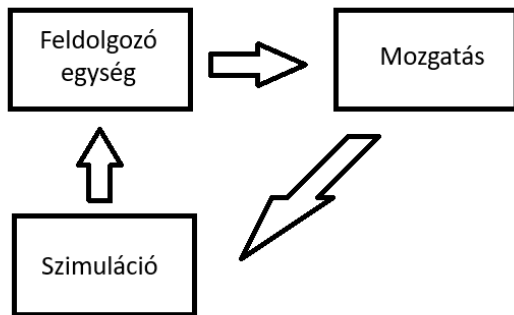
A drón végül a Tello egyszerűen csak "DJI Tello drón"-ja volt, külön plusz indok volt a választás mellett, hogy a Tello csinált egy komplett github repository-t hozzá, amiben benne volt például a drón modellje, de a valóságban történő mozgatásához szükséges node-ok is helyet kaptak benne. A tulajdonképpeni mozgatás nem volt benne, a node csupán azért felelt, hogy a drónnak küldött parancsokat legyen valaki, aki közvetíti a drón számára a parancsokat.

2. Feladat

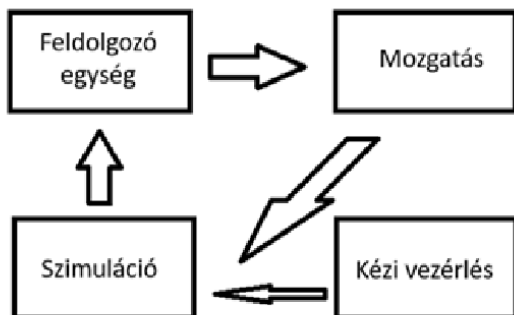
Az említett "DJI Tello drón"-hoz egy olyan programot előállítani, ami egy előre megadott szín sorrend szerint átrepül rajtuk, ha nem látja közvetlenül a következő karikát - akkor feltételezve, hogy nincs messze - egy kis területű keresést hajt végre. Fő programozási nyelvnek a python lett megbeszélve, ennek ellenére a drón github repository-jának eredeti node-jai CMake-ben lettek írva. További követelmény volt, hogy valamilyen formában az egyetemen is fejlesztett fuzzy-logikát is építsem bele. Mindezek összefogására pedig a csak Ubuntu (Linux disztribúció) operációs rendszeren futtatható ROS 2 (Robot Operation System/Robot Operációs Rendszer) legyen alkalmas. A tesztelésre szánt szimulációs program nem volt megbeszélve. A feladat elkészítésnek előzetesen a következő menetet szántam:

- Eszköz beszerzése, amin natív környezetben és hatékonyan tudok Ubuntut futtatni.
- Szimulátor és alap környezet felállítása.
- Szimulátorban a drón megjelenítése, a kamera képének olvasása.
- Kézzel mozgatás implementálása.
- Kamera képének feldolgozása.
- Automatikus logika implementálása.

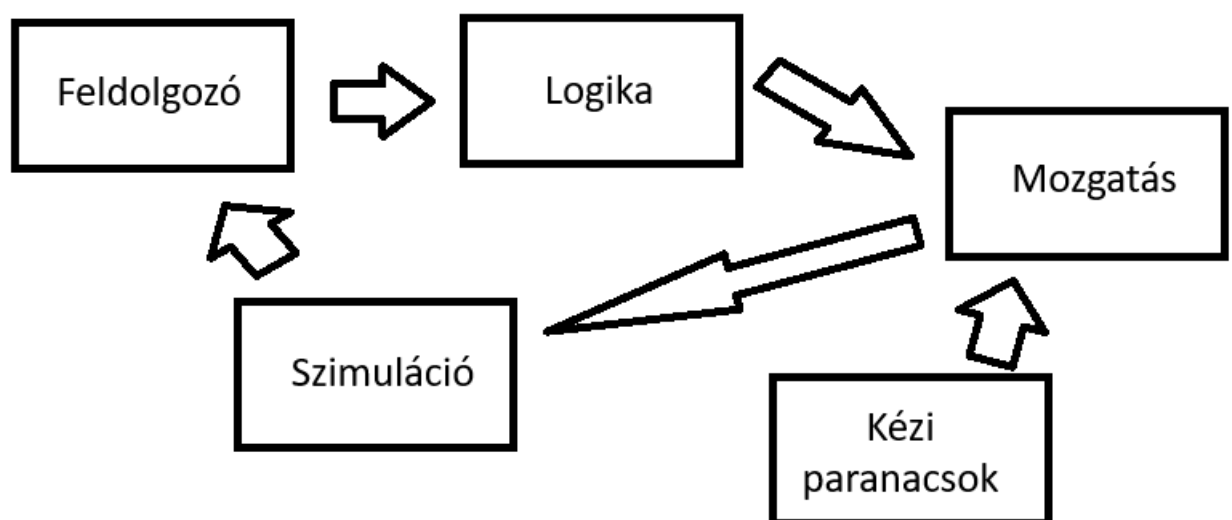
2.1. Tervezés



1. ábra. Az első terv vázlata



2. ábra. A bővített első terv



Mint ahogy látható bekerült egy "Mozgatás" egység, erre két indokból volt szükség: a logika és a kézi bevitel is mozgatni szeretné a drónt, így azokban nem kerül maga a mozgatás

megvalósításra, azok csak kiadják a parancsokat, és az az egy node ad csak utasításokat a drónnak. A második indok, között az összes apróbb tényező állt miszerint egyszerűbb volt így megvalósítani és így még az OOP alapelveknek is jobban megfelelt.

2.2. Tervezett node-ok

A várható node-ok így a következők lettek:

- Mozgatás: ez a node a szimuláció vagy a valóságban tartózkodó drón mozgatása (emelkedés süllyedés, előre mozdulás, oldalra fordulás és fordulás) ebben a node-ban zajlik. A kód a megfelelő paraméterek elindításával képes a valódi drónt is megmozdítani.
- Feldolgozó: valamilyen módszerrel a drón kameraképét feldolgozza, és megkeresi rajta a kör formákat, rendel hozzájuk egy szint, és kiszámolja, hogy mennyi az esélye, hogy ez azaz objektum, amit keresünk (Fuzzy logika itt került megvalósításra).
- Logika: itt történik a feldolgozó node által küldött adatok alapján a mozgás kiszámítása.
- Kézi parancsok: a terminálban az input olvasásával történik meg a mozgatás. Ez a node illetve a "Logika" node a "Mozgatás" node függvényeit hívja meg.

3. Fejlesztés

Mint mondtam már korábban is a fejlesztési terv, illetve a működés elgondolása sikeres volt, így ezek nem változtak a fejlesztés ideje alatt, csupán letisztultak.

3.1. Fejlesztő környezet beüzemelése

Első pontként szerettem volna egy natív Ubuntut. Mivel hordozható eszközzel nem rendelkeztem, és venni sem akartam, azért így egy olcsóbb és kompaktabb megoldást kerestem: egy külső SSD-t. A virtuális gépekkel (Virtual Machine) már volt tapasztalatom, de az nem volt pozitív, illetve egy külső hordozón is próbáltam már Ubuntut futtatni, mindkettő esetében ugyan az volt az észrevétel: lassú, akadozik és nem esik kézre. Végül egy 512 GB-os tárhelyű, de 1024 B/s átviteli sebességre képes SSD-t vettem - nyilván utóbbi csak USB 3.0 csatlakozás mellett teljesül.

Az Ubuntu verziójával egy kicsit már gondban voltam, ugyanis közben a drón repository-ját archiválták. Ez azért volt gond, mert a kódok és csomagok a ROS 2 Ubuntu Foxy elnevezésű verziójához voltak készítve, ami ezen a ponton már nem támogatott volt. A megoldás egyszerű volt, de ekkor még számomra ismeretlen: egy Docker készítése.

A Dockerek egy konténerplatformok, amely az Ubuntu operációs rendszeren futva lehetővé teszi az alkalmazások és azok összes függőségének különálló, hordozható egységekbe (konténerekbe) csomagolását. Ez azért jó, mert így a drón csomagjait nem kellett javítanom, vagy egyáltalán hozzáélnem, elég volt a Docker létrehozása.

Gyakorlatban ez úgy működött, hogy volt egy Ubuntun belüli Ubuntu, amit egy belső szerverként tudtam futtatni, és végeredményében ez futtat mindent a feladattal kapcsolatban.

3.2. Szimuláció

A következő pont talán a legnehezebb volt az összes közül. Mivel nem volt konkrétan megbeszélve, hogy melyik programban fogom a szimulációt, illetve a teszteket végezni, így ezt két programra szűkítettem le: a Gazebo és a Unity (utóbbi nem csak szimulációs program, de tökéletesen alkalmas arra is).

3.2.1. Program

A Gazebo pontos robotikai szimulációra és ROS-integrációra specializálódott, valósághű szenzoradatokat és összetett fizikai modellezést nyújt, míg a Unity egy sokkal szélesebb körű játék- és vizualizációs motor, amely kiemelkedő grafikát és vizuális minőséget ad, de a valósághű robotikai mozgáshoz további specifikus csomagokat igényel. Mind a kettő mellett szóltak érvek, a Gazebo használata mellett, hogy az külön ROS 2-höz lett tervezve, míg a Unity mellett, azaz egyszerű indok szólt, hogy azzal a programmal már bejáratos

voltam.

Végül ez a faktor döntött, és Unity-s szimulációs környezetet kezdtem meg felépíteni. Itt azonban hiba csúszott az előzetes várakozásaimba, ugyanis folyton folyvást falakba ütköztem. Először is a Unity-nek csak néhány verzióját, az úgynevezett LTS verzióknak (Long Term Support/Hosszú Távú Támogatottság) van csak Ubuntu támogatottsága (a Linux disztribúciók közül a Unity fejlesztői Ubuntura optimalizálják csak ezt a verziót). Már a letöltéssel is problémák voltak, arról nem is beszélve, hogy a hiányzó dokumentáció miatt nehezen lehet eldönteni, hogy most a Unity-t a Dockerbe, vagy azon kívülre kell telepíteni (a docker nagyon sok ideiglenes fájlal fut, ez későbbiekben is problémákat okozott), de a telepítés után az elindulás se ment zökkenő mentesen. Miután nagy nehezen a Unity Hub-ot, ami egy központi program a Unity-nek (itt lehet telepíteni magát a programot, hozzá assets-eket, de a projektek külső menedzselését is itt lehet végezni). Utána a bejelentkezés se ment zökkenő mentesen, mert a visszaigazoló ablakot nem akarta megnyitni. Ugyan voltak github repository-k, amik Unity számára tették lehetővé, hogy ROS 2 kódot beolvasson, és működtessen, de az egész nagyon instabil volt. Egyszer működött, a következő tesztnél már nem. Ezt a megközelítést lassan feladtam. Többször is próbálkoztam, hogy Unity-vel szimuláljak, de minden próbálkozásnál egyre kevésbé sikerült mozgásra bírnom.

Közben és végül a Gazebo nevű programot kezdtem el használni, amiről ugyan tudtam, hogy beépítetten képes a ROS 2 integrációra, ugyanakkor ez volt az összes információm róla. Mivel ez egy szigorúan Ubuntura tervezett program volt (nincs is hivatalos Windows-os verziója), így a telepítés gyerekjáték volt. Ami már kevésbé volt gyerekjáték, hogy a ROS 2 és a Gazebo között beüzemeljem a hidat. Ennek elsősorban az volt az oka, hogy a dokumentációk már a frissebb verziókról szóltak, így téves információim voltak.

```
<plugin name='gazebo_ros_init' filename='libgazebo_ros_init.so'>
<plugin name='gazebo_ros_factory' filename='libgazebo_ros_factory.so'>
<plugin name='gazebo_ros_state' filename='libgazebo_ros_state.so'>
```

A harmadik sor nélkül nem tudja a leidezett drón topicját publikálni, amik viszont létfontosságú adatok minden szempontból. A harmadik sorra nincs szükség az újabb verziókban, mert a másik két csomag már tartalmazza, ellenben Foxy verzión még nem.

3.2.2. Szimulációs környezet

Megbeszélés szerint nekem a szimulációban szükségem volt három körre különböző színben, hogy a drón tudjon valamit keresni. A gondot ezúttal az okozta, hogy a Gazebo-ban csak az alapformák találhatók meg (kocka, gömb, piramis alak), és semmilyen szerkesztő nincs hozzá, sőt, ha színt akarok állítani azt is csak a .sdf vagy. world fájl módosításával tudok. A megoldás egyszerűbb lett, mint azt ez után vártam volna: egy fánk alakú objektumot letöltöttem az internetről, aztán ezt. dae fájlal alakíttattam, majd ennek kellett

egy keret rendszert csinálni, és azt beolvasni a Gazebo világba. Itt is ment egy kis időhúzás, ugyanis a Dockerben, ahonnan hivatalosan beolvassa a modelleket, az a Dockerben az egy ideiglenes mappa, és a bezárása után törlődik a mappa és annak teljes tartalma. A megoldás csak szimplán annyi lett, hogy az én munka mappámba irányítottam át a .sdf fájlban, hogy hol vannak a fánkok modelljei.

Maga a Gazebo világ: `drone_ws/sim/worlds/donuttest.sdf`

A fánkok helye: `donuts/`(és itt a három szín szerint elnevezve)



3.3. Kézi vezérlés

Első lépésként a már sokat emlegetett kézi vezérlést akartam megcsinálni. Ez azért bizonyult fontosnak később, mert a mozgást vezérlő függvények először itt voltak megvalósítva, majd ezeket később kiemeltem ebből a node-ból.

A kézi bevitelért egy beolvasó függvény felel: Látható, hogy a "c" változó megkapja a terminálba írt karaktert, majd egy sok else-if-et használva kiválasztja a megfelelő műveletet (switch-case nincs a python-ban).

```
#Main control loop that reads keyboard input and executes drone movements
def run(self):
    while rclpy.ok():
        c = getch()
        if c == 'w': self.controller.step_x(+0.1)
        elif c == 's': self.controller.step_x(-0.1)
        elif c == 'a': self.controller.step_y(+0.1)
        elif c == 'd': self.controller.step_y(-0.1)
        elif c == ' ': self.controller.step_z(+0.1)
        elif c == 'c': self.controller.step_z(-0.1)
        elif c == 'q': self.controller.step_yaw(+0.05)
        elif c == 'e': self.controller.step_yaw(-0.05)
        elif c == 'x': break
```

```
#Moves the drone forward/backward
def step_x(self, dx: float):
    if self.simulation:
        req_g = GetEntityState.Request()
        req_g.name = self.model_name
        req_g.reference_frame = self.reference_frame
        fut_g = self.cli_get.call_async(req_g)
        rclpy.spin_until_future_complete(self, fut_g)

        if not fut_g.result() or not fut_g.result().success:
            self.get_logger().error("GetEntityState unsuccessful.")
            return

        pose: Pose = fut_g.result().state.pose
        twist: TwistMsg = fut_g.result().state.twist
        q = pose.orientation
        q_w, q_x, q_y, q_z = q.w, q.x, q.y, q.z
        current_roll, current_pitch, current_yaw = tfe.quat2euler([q_w, q_x, q_y, q_z], axes='sxyz')
        yaw = current_yaw

        dx_body = dx
        dy_body = 0.0

        dx_world = math.cos(yaw) * dx_body - math.sin(yaw) * dy_body
        dy_world = math.sin(yaw) * dx_body + math.cos(yaw) * dy_body

        pose.position.x += dx_world
        pose.position.y += dy_world

        req_s = SetEntityState.Request()
        req_s.state = EntityState()
        req_s.state.name = self.model_name
        req_s.state.reference_frame = self.reference_frame
        req_s.state.pose = pose
        req_s.state.twist = twist

        fut_s = self.cli_set.call_async(req_s)
        rclpy.spin_until_future_complete(self, fut_s)

        if not fut_s.result() or not fut_s.result().success:
            self.get_logger().error("SetEntityState unsuccessful.")
    else:
        vx = dx / abs(dx) * self.lin_speed if dx != 0 else 0.0
        self.send_cmd(vx, 0.0, 0.0, 0.0)
        self.get_logger().info(f"Sent forward velocity: {vx:.2f} m/s")
```

A `step_x` függvény a drón előre vagy hátra mozgatásáért felel. Két módban működik:

1. Szimulációs mód

- Lekérdezi az aktuális pozíciót és orientációt a Gazebo szimulátorból.
- Kinyeri a yaw szöveget (a drón elfordulása) a kvaterniók Euler-szögekké konvertálásával.
- Koordináta-transzformáció.
- A test-koordináta rendszerben az előre-mozgatás az X tengely mentén történik.
- Az alábbi képlet segítségével átalakítja világkoordináta-rendszerre:
- Frissíti a pozíciót: `pose.position.x += dx_world` és `pose.position.y += dy_world`
- Elküldi az új állapotot a Gazebo szimulátornak

2. Valós drón mód

- Sebességet számol: `vx = ±lin_speed`
- Elküld egy Twist üzenetet a `/cmd_vel` topicra
- Naplózza a sebességet

A lényeg, hogy a drón haladási irányát mindig a jelenlegi orientációjához (yaw-hoz) igazítja, így az relatív mozgatas lesz. Ez azért fontos, mert eleinte úgy volt megcsinálva, hogy a drón nem a saját koordináta rendszere szerint mozdult, hanem minden esetben a világ koordináta rendszere szerint, így hiába fordult el.

Ez alapján működik az összes többi függvény is.

3.4. Kamerakép elemzése

3.4.1. Hough körök

Nagyon hamar felfedeztem magamnak a Hough-transzformáció alapú kördetektor metódust. A Hough Circle Detector egy olyan képfeldolgozó algoritmus, amelynek célja, hogy egy képből megbízhatóan és hatékonyan megtalálja a kör alakú objektumokat, még akkor is, ha azok részben takartak, zajosak vagy nem tökéletes alakúak. Fontos kijelenteni, hogy ez nem szín, hanem logikai alapú keresés.

Működése

a technika egy akkumulátortér (accumulator space) nevű speciális memóriaterületet használ.

- Ahelyett, hogy megpróbálná azonnal azonosítani a teljes kört, minden egyes képelen (edge) lévő pont szavaz.
- Minden egyes élpont megszavazza, hogy hol lehet a kör középpontja (x, y) és mekkora lehet a sugara (r).

- A legtöbb szavazatot kapó (x, y, r) kombinációt (a csúcsokat) tekinti a program azonosított körnek.

Ez egy nagyon jó algoritmus, főleg mert a kép zaj nem zavarja, és akár nem teljesen a képen lévő köröket is képes megtalálni.

Megvalósítás

```
#Detects circles in a grayscale image using Hough Circle Transform
def detect_circles(gray, dp=1.2, min_dist=50, param1=100, param2=40, min_radius=20, maxR=
    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=dp, minDist=min_dist,
    | | | | | | | param1=param1, param2=param2, minRadius=min_radius, maxR
    if circles is None:
    |     return []
    circles = np.uint16(np.around(circles))
    return circles[0, :].tolist()
```

A Hough kör kereső algoritmus a `detect_circles` függvényben van megvalósítva: ebben hívódik a `cv2.HoughCircles`. Ez a függvény adja vissza a talált körök koordinátáit és sugarát (ha nincs találat, üres lista).

```
#Calculates color fraction in the ring area of a detected circle
def color_fraction_in_circle(bgr_img, circle, color_ranges_hsv):
    x, y, r = int(circle[0]), int(circle[1]), int(circle[2])
    h, w = bgr_img.shape[:2]

    mask = np.zeros((h, w), dtype=np.uint8)
    ring_width = max(5, int(r * 0.3))
    cv2.circle(mask, (x, y), r, 255, thickness=ring_width)
    hsv = cv2.cvtColor(bgr_img, cv2.COLOR_BGR2HSV)
    best_color = None
    best_fraction = 0.0

    for name, ranges in color_ranges_hsv.items():
        mask_color = np.zeros_like(mask)
        for low, high in ranges:
            low_np = np.array(low, dtype=np.uint8)
            high_np = np.array(high, dtype=np.uint8)
            m = cv2.inRange(hsv, low_np, high_np)
            mask_color = cv2.bitwise_or(mask_color, m)

        keep = cv2.bitwise_and(mask_color, mask)

        count_color = int(cv2.countNonZero(keep))
        count_circle = int(cv2.countNonZero(mask))
        frac = (count_color / count_circle) if count_circle > 0 else 0.0

        if frac > best_fraction:
            best_fraction = frac
            best_color = name

    return best_color, clamp01(best_fraction)
```

A `color_fraction_in_circle` függvény elemzi minden talált kör környékét: létrehoz egy gyűrűmaszkot (mask) a kör körül, a képet HSV-re konvertálja és a `self.color_ranges_hsv` színintervallumait használva készít bináris maszkokat, a gyűrű maszk és a színmaszk logikai ÉS-e adja meg, mennyi pixelen van az adott szín a gyűrűben, visszaadja a legjobb (legnagyobb arányú) színt és a hozzá tartozó arányt (0-1 között).

Utóbbi arány reprezentálja a fuzzy logika jelenlétét,

ugyanis ha több egyszínű kört is érzékel, ezek alapján az értékek alapján dönt.

```

def image_callback(self, msg: Image):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    except CvBridgeError as e:
        self.get_logger().error(f'CvBridge error: {e}')
        return

    small = cv2.resize(cv_image, (cv_image.shape[1]//1, cv_image.shape[0]//1))
    gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
    gray = cv2.medianBlur(gray, 5)
    circles = detect_circles(gray)
    scale_x = cv_image.shape[1] / float(small.shape[1])
    scale_y = cv_image.shape[0] / float(small.shape[0])
    scaled = []

    for c in circles:
        x = int(c[0] * scale_x)
        y = int(c[1] * scale_y)
        r = int(c[2] * ((scale_x + scale_y) / 2.0))
        scaled.append((x, y, r))

    count = len(scaled)
    presence_degree = clamp01(count / max(1, self.expected_max))

    detections = []
    max_color_degree = 0.0

    #Analyze each detected circle's color
    for c in scaled:
        best_color, frac = color_fraction_in_circle(cv_image, c, self.color_ranges_hsv)
        detections.append({
            'x': c[0],
            'y': c[1],
            'r': c[2],
            'color': best_color,
            'fraction': frac
        })

        if frac > max_color_degree:
            max_color_degree = frac

    activation_degree = min(presence_degree, max_color_degree)
    activated = activation_degree >= self.threshold

    self.activation_pub.publish(Bool(data=activated))
    self.score_pub.publish(Float32(data=float(activation_degree)))
    self.circles_pub.publish(String(data=json.dumps(detections)))

```

A kép-

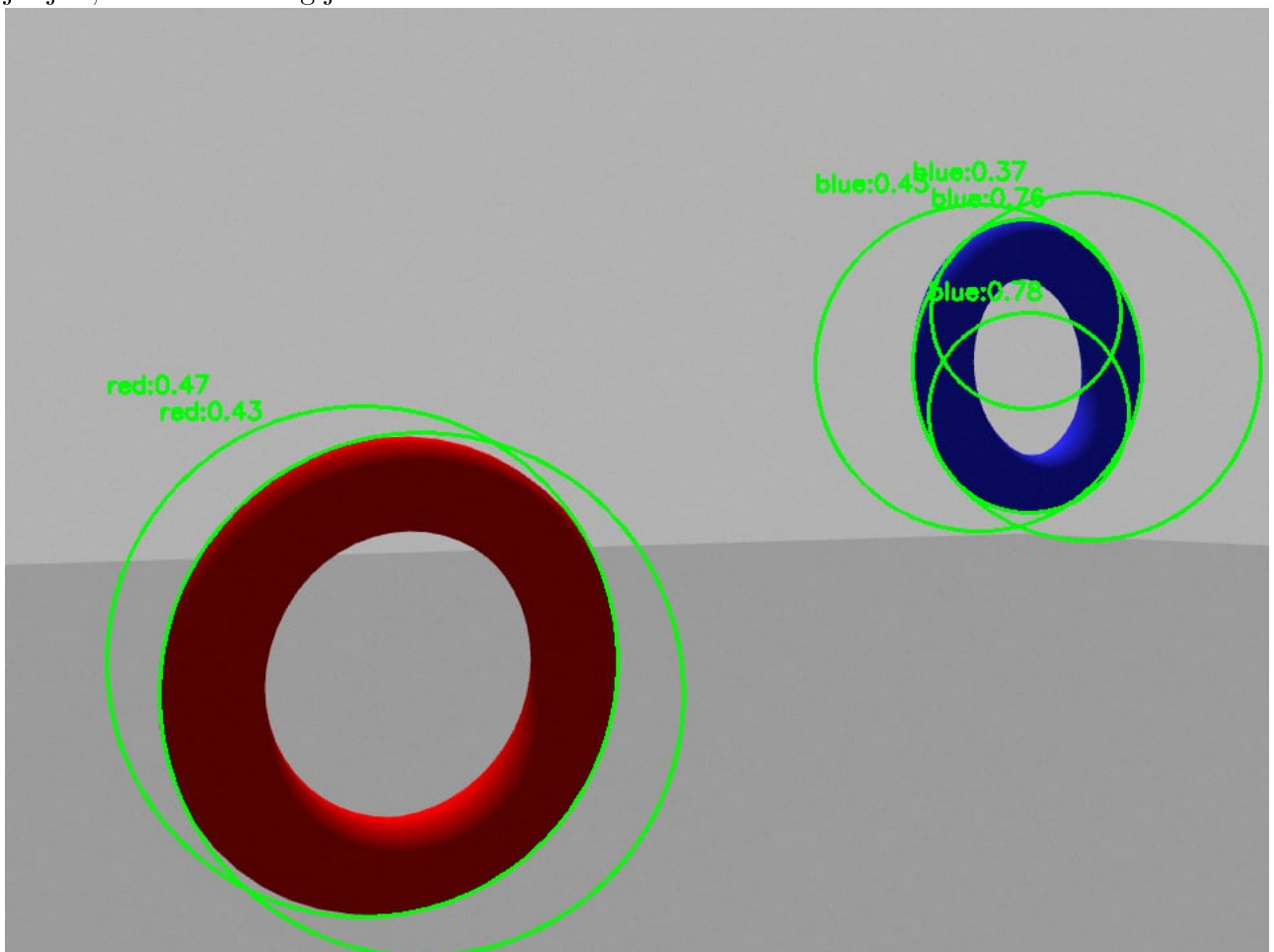
feldolgozás főhelye az image_callback metódus:

- A nyers üzenetet CvBridge-pel BGR képpé (cv_image) alakítják.
- A small = cv2.resize-zal opcionálisan kicsinyítik.
- Szürke színskála szerinti konverzió: Gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY).
- Zajcsökkentés: gray = cv2.medianBlur(gray, 5).
- Majd erre a képre meghívódik a circles = detect_circles(gray).

Visszaskalázás az eredeti képre:

- Mivel a Hough a small képen fut, az `image_callback` kiszámolja a skálát.
 - `scale_x = cv_image.shape[1] / float(small.shape[1])`
 - `scale_y = cv_image.shape[0] / float(small.shape[0])`
- A kör koordinátáit és sugarát visszaskalázza az eredeti képméretre, és scaled listába gyűjti.

Ezekután már csak az rész van, ami azért felel, hogy az információkat topic-on keresztül kiírja, illetve az egésznek csináltam vizualizációt is, egyrészt mert a tesztelésnél sokszor jól jön, másrésztől meg jól néz ki:



3.4.2. Kontúr alapú körkeresés

Probléma

A fenti kép több szempontból is jó, főleg azért, mert lehet rajta látni, hogy van olyan állapota, amikor jóval több kört talál, mint kéne. Bár ezek ugye súlyozva vannak, de az érzékelő megzavarodik tőlük. Éppen ezért kellett egy alternatív érzékelő is, mert a Hough kör detektor megbízhatatlan volt.

Megoldás

Általános ötlet a következő volt: színalapú maszkolás, majd morfológia, aztán kontúrok keresése és minEnclosingCircle sugara alapján értékelés. Végül pedig fuzzy logikával súlyozott bizalom.

```
#Analyzes contours in a mask and returns detected circles with confidence scores
def analyze_mask_contours(self, bgr_img, mask, color_label_hint, expected_hue):
    kernel = np.ones((5,5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    detections = []
    H, W = bgr_img.shape[:2]
    image_area = W * H
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 250:
            continue
        (x,y), r = cv2.minEnclosingCircle(cnt)
        if r < 8 or r > max(W,H):
            continue

        mask_zero = np.zeros_like(mask)
        cv2.drawContours(mask_zero, [cnt], -1, 255, -1)
        hsv = cv2.cvtColor(bgr_img, cv2.COLOR_BGR2HSV)
        mask_idx = np.where(mask_zero > 0)
        if mask_idx[0].size == 0:
            continue
        mean_h = float(np.mean(hsv[mask_idx][:,0]))
        mean_s = float(np.mean(hsv[mask_idx][:,1]))
        mean_v = float(np.mean(hsv[mask_idx][:,2]))
        f_size = max(fuzzy_ideal_radius(r), fuzzy_small_radius(r)*0.5, fuzzy_large_radius(r)*0.3)
        f_color = fuzzy_color_strength((mean_h, mean_s, mean_v), expected_hue)
        confidence = clamp01(0.55 * f_size + 0.45 * f_color)

        #Only consider detections with significant confidence
        detections.append({
            'x': int(x), 'y': int(y), 'r': int(r),
            'area': float(area),
            'color_hint': color_label_hint,
            'mean_hsv': [mean_h, mean_s, mean_v],
            'confidence': float(confidence)
        })
    return detections
```

Előfeldolgozás:

- BGR skálát HSV alakítjuk.
- build_color_masks(hsv) létrehozza színmaszkokat az előre definiált HSV tartományokból.
- Maszk tisztítás és kontúrkeresés.
- morfológiai zárás.
- kontúrok keresése.

- kis területű kontúrok eldobása.
- kör becslése.
- levágás, vagyis kis vagy túl nagy sugarú kontúrok kihagyása.

```

#Main image processing callback
def image_callback(self, msg: Image):
    try:
        cv_img = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    except CvBridgeError as e:
        self.get_logger().error(f"CvBridgeError: {e}")
        return

    hsv = cv2.cvtColor(cv_img, cv2.COLOR_BGR2HSV)
    masks = self.build_color_masks(hsv)
    #Special handling for red
    red_mask = None
    if 'red' in masks and 'red2' in masks:
        red_mask = cv2.bitwise_or(masks['red'][0], masks['red2'][0])
        masks.pop('red2', None)
        masks['red'] = (red_mask, masks['red'][1], 'red')
    all_detections = []

    #Analyze each color mask for contours
    for name, (mask, hue, label) in masks.items():
        dets = self.analyze_mask_contours(cv_img, mask, label, hue)
        for d in dets:
            d['color'] = label
            all_detections += dets

    #Select best detection per color
    best_by_color = {}
    for d in all_detections:
        c = d['color']
        if c not in best_by_color or d['confidence'] > best_by_color[c]['confidence']:
            best_by_color[c] = d
    publish_list = []
    for c, d in best_by_color.items():
        publish_list.append({
            'x': d['x'], 'y': d['y'], 'r': d['r'],
            'color': c, 'fraction': d['confidence'], 'area': d['area']
        })

    #Calculate overall presence degree
    best_conf = 0.0
    total_area_ratio = 0.0
    img_area = cv_img.shape[0] * cv_img.shape[1]
    for p in publish_list:
        if p['fraction'] > best_conf:
            best_conf = p['fraction']
        total_area_ratio += (p['area'] / img_area)
    presence_degree = clamp01(min(1.0, total_area_ratio * 50.0) * 0.6 + best_conf * 0.4)

    activated = presence_degree >= self.threshold

```

Kiszámolják a kontúr belsejében lévő pixelek HSV átlagát (mean_h, mean_s, mean_v). Aztán jöhet a fuzzy értékek (fuzzy... függvények) ezek adnak tagsági értékeket a mérethez.

Ezután a szín erejét is meghatározom a `fuzzy_color_strength` függvénnyel, ami összeveti a középértékű hue-t az elvárt hue-val, és figyelembe veszi s és v komponenseket.

Összeadjuk `confidence = clamp01` függvénnyel (méret és a szín kombinálva).

Kiválasztás és összesítés: minden színhez megtartják a legjobb detektálást. A `presence_degree` számítása: a `publish_list` elemeinek területeinek összege és a legjobb confidence súlyozása.

Ezután már csak a publikálás és a vizualizáció van hátra.

Összegzés

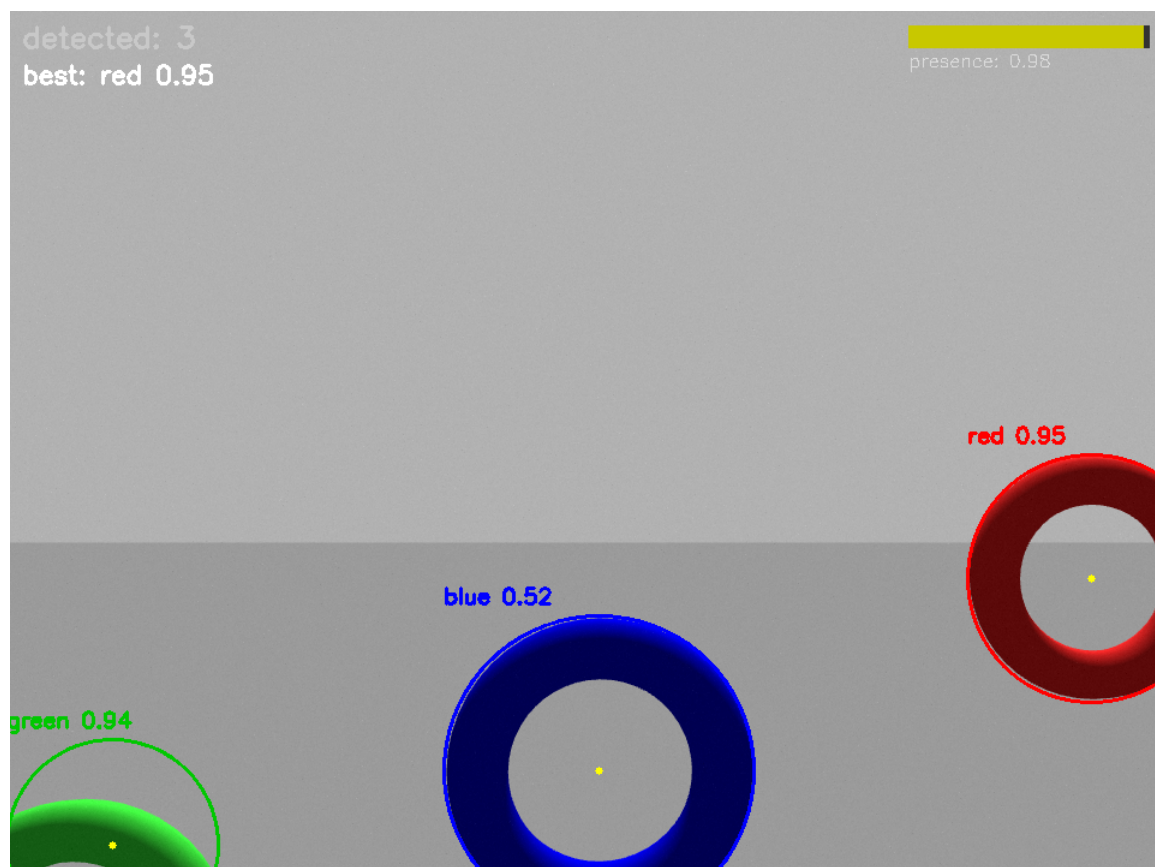
Azért volt szükség kettő körkeresőre is, mert bizonyos esetekben az egyik jobban működik, mint a másik. Összehasonlítás képpen:

Hough kör kereső:

- Előny: közvetlenül körformákat keres.
- Hátrány: többszörös méretek vagy a részben fedett körök problémát jelentenek.

Kontúr szín érzékelő:

- Előny: robusztusabb szín és terület értékelés, képes kezelni szabálytalanabb, törött vagy részben takart alakzatokat.
- Hátrány: színmaszkolás hibáknál, sok hamis pozitív/negatív lehet.



3.5. Autómatizált kör keresése

Készítésnél közös megegyezés szerint nem a ROS 2 beépített útvonal keresőjét akartam használni. Helyette belefutottam az úgy nevezett PID útvonalkövetés logikába.

3.5.1. A PID logika

A "PID útvonalkeresés" valójában útvonalkövetést jelent, mivel a PID (Proportional-Integral-Derivative) szabályzó egy vezérlőmechanizmus, és nem egy útvonalat kereső algoritmus. A szerepe az, hogy a már kiszámított útvonalat a robot pontosan és stabilan kövesse. A PID szabályzó célja a hiba (error) minimalizálása, ami a célpont és a mért aktuális érték közötti különbség. A szabályzó ezt a hibát használja fel egy korrekciós jel létrehozására, három komponens alapján:

- P (Proportional / Arányos): Ez a tag a pillanatnyi hiba nagyságával arányosan generál korrekciót.
- I (Integral / Integráló): Ez a tag összegzi az idő múlásával felgyülemlett hibát. A szerepe az állandósult hiba megszüntetése.
- D (Derivative / Deriváló): Ez a tag a hiba változási sebességére reagál, ezzel csillapítja a kilengéseket és a rendszert stabilizálja.

Összefoglalva: Az útvonalkövetésnél a PID felel azért, hogy a már kiszámított útvonalon a drónod finoman, pontosan és stabilan maradjon, miközben folyamatosan korrigálja a külső zavarokat és a mozgás dinamikáját.

3.5.2. Megvalósítás

`cb_activation(self, msg)`: folyamatosan olvassa a detektort.

`cb_circles(self, msg)`: a kapott `msg.data`-t parse-olja. Csak bizonyos sugáron belüli detektálásokat tartja meg.

A végén meghívja a `self.control()` metódust, tehát minden új kör-információra azonnal futtatja a vezérlési logikát.

```
#Update activation flag from detector
def cb_activation(self, msg):
    self.active = msg.data

#Update activation degree from detector
def cb_degree(self, msg):
    self.degree = msg.data

#Parse detected circles JSON and run control loop
def cb_circles(self, msg):
    try:
        text = msg.data.strip()
        loaded = json.loads(text)

        self.circles = [
            c for c in loaded
            if 20 < c.get("r", 0) < 500
        ]

    except Exception as e:
        self.get_logger().warn(f"JSON parse error: {e}")
        self.circles = []

    self.control()
```

```

#Main control logic
def control(self):
    #If finished the sequence, hover
    if self.current_index >= len(self.color_order):
        self.get_logger().info("All toruses completed! Hovering.")
        self.controller.send_cmd(0.0, 0.0, 0.0, 0.0)
        return

    target_color = self.color_order[self.current_index]

    #Handle in-progress rotation step (180-degree rotation)
    if self.rotation_steps > 0:
        self.controller.step_yaw(0.5)
        self.rotation_steps -= 1
        self.get_logger().info(f"Rotating 180 degrees... remaining: {self.rotation_steps}")
        if self.rotation_steps == 0:
            self.get_logger().info("180 degree rotation complete.")
            return

    #Handle pass-through phase where drone just moves forward
    if self.pass_counter > 0:
        self.controller.step_x(0.18)
        self.controller.step_y(0.0)
        self.controller.step_z(0.0)
        self.controller.step_yaw(0.0)

        self.pass_counter -= 1
        if self.pass_counter == 0:
            self.get_logger().info(f"Completed {target_color} torus, finished pass-through.")
            self.current_index += 1
            if self.current_index < len(self.color_order) and self.current_index % 2 == 1:
                self.rotation_steps = self.rotation_duration
            return

    target_circles = [c for c in self.circles if c.get("color") == target_color]

```

Ha a sorrendet befejezte, majd hover pozi-

cióba állítja a drónt.

Cél szín kiválasztása.

Fordulás, ha végzett a Pass-through fázissal

Pass-through fázis, ez azt jelenti, hogy a kör átmérőjét nagyon nagynak látja, vagyis közel van a cél, így elindul előre.

Cél detekciók szűrése.

```

#If no detections, perform search behaviour
if len(target_circles) == 0:
    #If we recently saw a very close target, force pass-through
    if self.last_known_radius > 200:
        self.get_logger().info(f"Target lost but close (r={self.last_known_radius:.1f}). Forcing pass-through.")
        self.controller.send_cmd(0.0, 0.0, 0.0, 0.0)
        self.pass_counter = self.pass_steps
        self.last_known_radius = 0.0
        return

    #Switch between rotate and forward search states
    if self.search_step_duration <= 0:
        if self.search_state == 'rotate':
            self.search_state = 'forward'
            self.search_step_duration = self.search_forward_duration
            self.get_logger().info("Search: Target lost. Moving forward.")
        else:
            self.search_state = 'rotate'
            self.search_step_duration = self.search_rotation_duration
            self.get_logger().info("Search: Target lost. Rotating.")
            self.search_height_dir *= -1.0
            dz = self.search_height_step * self.search_height_dir
            self.controller.step_z(dz)
            self.get_logger().info(f"Search: Adjusting height by {dz:.2f} m.")

    self.search_step_duration -= 1
    step_x = 0.0
    step_yaw = 0.0

    if self.search_state == 'rotate':
        step_yaw = self.search_yaw_rate
    elif self.search_state == 'forward':
        step_x = self.search_forward_speed
    self.controller.step_x(step_x)
    self.controller.step_y(0.0)
    self.controller.step_z(0.0)
    self.controller.step_yaw(step_yaw)
    self.last_known_radius = 0.0
    return

```

Ha nincs detektált cél:

Ha korábban nagyon közel volt, akkor lefut egy kényszer pass-through, ez a rész azért kell, mert néha a drón úgy lép előre, hogy eltűnik a kameráról a fánk (túl közel van és már egyik kör kereső sem talál kört), így elindul az utolsó ismert vektoron előre- Egyébként neki áll keresni egy kis területen belül. Körbefordul, majd kicsit arrébb megy, és újra körbe fordul. Ezt ismétli egy kis négyzet körvonalán, közben néha emelkedve.

```

#Reset search bookkeeping and pick the best donut
self.missing_counter = 0
self.search_step_duration = 0
self.search_state = 'rotate'

torus = max(target_circles, key=lambda c: c.get("r", 0))

cx = float(torus.get("x", self.img_center_x))
cy = float(torus.get("y", self.img_center_y))
r = float(torus.get("r", 0))

self.last_known_radius = r

#If close enough, initiate pass-through
if r >= self.radius_threshold_max:
    self.controller.send_cmd(0.0, 0.0, 0.0, 0.0)
    self.pass_counter = self.pass_steps
    self.last_known_radius = 0.0
    self.get_logger().info(f"{target_color} donut: close enough (r={r:.1f}), starting pass-through.")
    return

#Compute image-plane errors and desired correptions
error_x = cx - self.img_center_x
error_y = cy - self.img_center_y
step_yaw = -error_x * self.k_yaw
step_y = 0.0
step_z = -error_y * self.k_z
step_x_min = 0.08
if r > 180:
    step_x_min = self.forward_step

```

Ha van detektált cél:

Először alaphelyzetbe állítja a kereső paramétereit.

Kiválasztja a legnagyobb fánkot. Kinyeri a középpontot és sugarat. Frissíti az utolsó ismert sugarat. Ha elég közel a cél akkor elindítja az átmenő blokk részeit, és elindul előre.

Ha eddig eljutott a kód, akkor nekiáll a hiba számításoknak:

$\text{step_yaw} = -\text{error_x} * k_yaw$ (horizontális eltérés alapján forgatáskorrekció)

$\text{step_z} = -\text{error_y} * k_z$ (vertikális eltérés alapján magasságkorrekció)

```

#If well-aligned, go at max forward speed; otherwise take smaller corrective steps
ALIGN_TOL = 30
if abs(error_x) < ALIGN_TOL and abs(error_y) < ALIGN_TOL:
    step_x = self.max_step_x
    step_yaw = 0.0
    self.get_logger().info(f"Aligned. Moving towards {target_color} torus. Speed: {step_x:.2f} m/s (MAX SPEED)")
else:
    step_x = step_x_min
    self.get_logger().info(f"Alignment required: dx={error_x:.0f}, dy={error_y:.0f}. Correcting.")

#Clamp command magnitudes to configured maxima
step_x = float(np.clip(step_x, -self.max_step_x, self.max_step_x))
step_y = float(np.clip(step_y, -self.max_step_y, self.max_step_y))
step_z = float(np.clip(step_z, -self.max_step_z, self.max_step_z))
step_yaw = float(np.clip(step_yaw, -0.15, 0.15))

#Send motion commands to the controller
self.controller.step_x(step_x)
self.controller.step_y(step_y)
self.controller.step_z(step_z)
self.controller.step_yaw(step_yaw)

self.get_logger().info(
    f"Tracking {target_color}: dx={step_x:.2f}, yaw={step_yaw:.3f}, r={r:.1f}"
)

```

Előrehaladás logika:

Van egy `ALIGN_TOL = 30` pixel tolerancia: ha a hiba mindkét tengelyen kisebb ennél, akkor gyors előrehaladás és nulla fordulás. Egyébként kisebb korrekció.

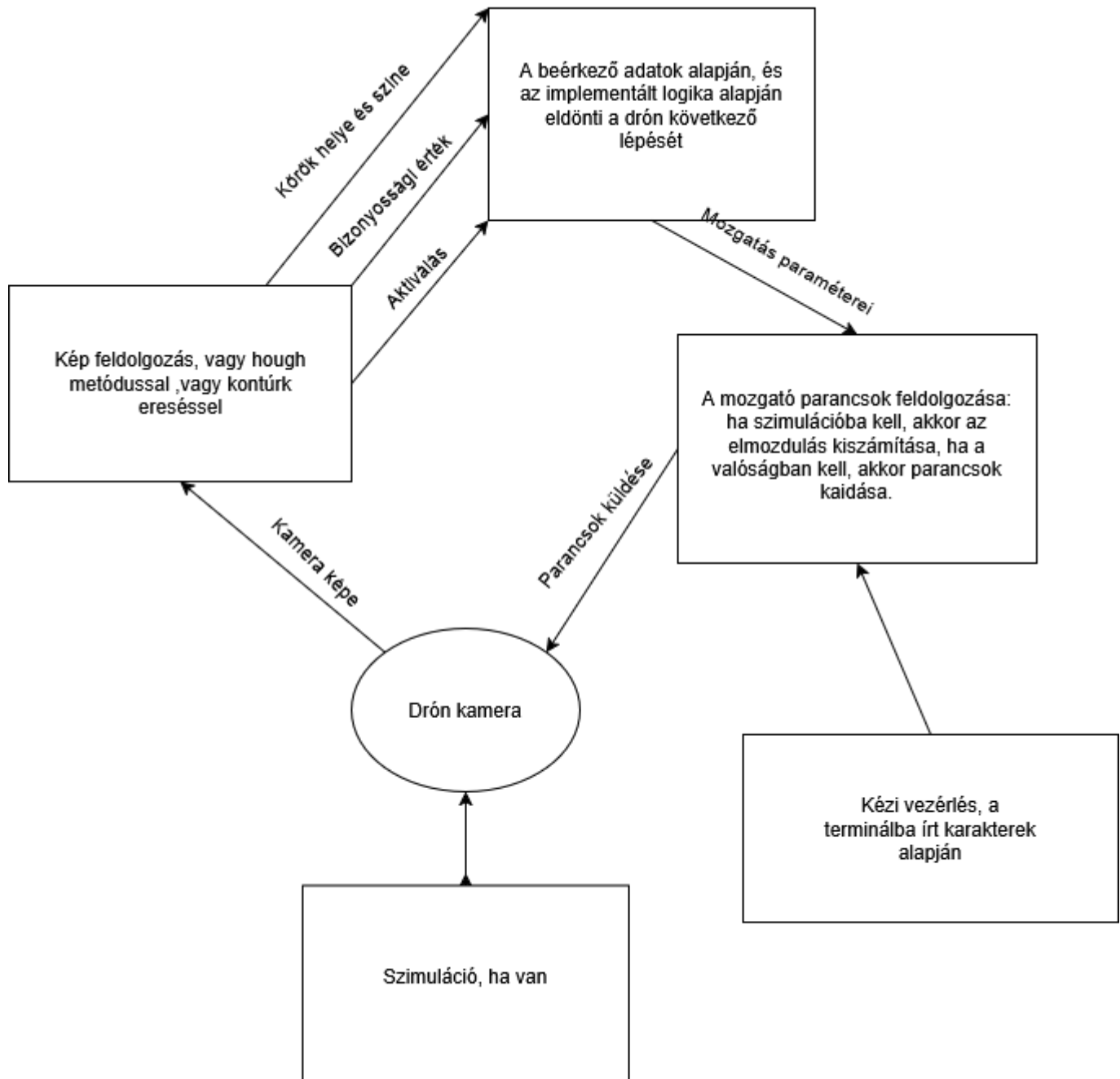
Ha $r > 180$ akkor előre fogunk menni. A parancsok korlátozzuk. A `np.clip`-pel korlátozza `step_x/step_y/step_z` a `max_step_*` értékekre, és `step_yaw`-re.

Parancsok elküldése a controllernek.

Végül kiírja a jelenlegi követési státuszt.

4. Működése

A kialakított rendszer célja, hogy a Tello drón vezérlése, a kamera képének feldolgozása és a szimulációs környezet közötti adatkapcsolat átlátható és valós időben működő keretet biztosítson. A teljes működés ROS 2 alapú kommunikációra épül, amely egységes felületet nyújt a valódi drón, a Gazibo szimuláció és a felhasználói terminál között.



4.1. A működés folyamata

A rendszer indítása után a ROS 2-ben futó `tello_driver` csomag automatikusan felépíti a kapcsolatot a drónnal, és elkezd publikálni a kamera képkockákat az `/image_raw` vagy a névterezett (`topic_ns/image_raw`) topicon. Ezzel párhuzamosan a vezérlő node (kézi

vagy automatikus) sebességparancsokká alakítja, és elküldi a `/cmd_vel` topicon.

4.2. A felhasználó által látható elemek

- Valós idejű kamerakép.
- A drón pozíciójának és mozgásának megjelenítése, amely folyamatosan követi a ROS 2-ben kapott sebességparancsokat.
- Terminál-logok, amelyek jelzik a kapcsolat felépülését, esetleges hibákat, valamint a kör detektorok által feldolgozott képinformációkat.
- A kamera kép könnyen konvertálható ROS image formátumba.
- A kördetektor modul használatakor a színes körök a kameraképen jól láthatóak túlszaturált fényviszonyok mellett is.

4.3. Tapasztalt problémák és hibajelenségek

- A camera topic névtér-problémája: előfordult, hogy a kör detektor nem kapta meg a kameraképet, mert a driver más névtérrel publikálta, mint amit a detektor figyelt.
- Ilyenkor a logban nem jelent meg semmilyen detektálási kísérlet. Docker-es környezet X11-támogatási hibái: a GPU-t vagy megjelenítést használó konténer indításánál előfordult, hogy az `XDGL_SESSION_TYPE` nem adott értéket, ami X11/GPU megosztási problémákhoz vezetett. Nem eredményez hibát vagy összeomlást, mert a Gazebo átáll CPU-s megjelenítésre.
- Képkésés időnként nagyobb CPU-terhelés mellett: amikor a kamera kép + szimuláció + kör detektor egyszerre fut, a képfrissítés néha akadozott különösen, ha a Dockerben nem lett megfelelően átadva a GPU. Ilyenkor maga a terminál is beakad, és ilyenkor hajlamos a PID számítás félremenni. Nem gyakori hiba, de a drón könnyen elvesztheti a célt ilyenkor.
- A kör detektor modul nem jelez vissza semmit: még akkor sem, amikor a kameraképen jól láthatóak voltak a körök. Ez jellemzően akkor fordult elő, ha:
 - A képernyőfelbontást a driver más formátumban adta vissza.
 - Nem volt meg az OpenCV támogatás a konténerben.
 - A topik nevek eltértek a vártól.

4.4. Működés körülmények

Az egész végrehajtható csak `start.bash` fájl alkalmazásával.

A rendszer helyesen működik, ha:

- minden ROS 2 node ugyanabban a névtérben vagy megfelelően konfigurált bridge-en keresztül kommunikál.
- a Docker konténerhez helyesen van csatolva az X11 és a GPU.
- a drón stabil WiFi kapcsolaton keresztül érhető el.
- a kö detektor modult a pontos topiknévvel indítják.

Helytelen működés valószínű, ha:

- a video streamet a konténer nem kapja meg (X11/GPU-hiba).
- a képformátum OpenCV-ben nem kompatibilis.
- túl nagy CPU-használat miatt a kamera framerate leesik.
- eltérő névterek miatt node-ok nem találják egymást.

5. Fejlesztési tervek

A szakdolgozat során sikerült kialakítani egy működő rendszert, amelyben a Tello drón ROS 2 környezetből vezérelhető, a kamera kép valós időben feldolgozódik, majd a mozgás a szimulációs platformon is megtörténik. Bár a rendszer a tervezett funkciók nagy részét megvalósítja, véleményem szerint még bőven van potenciál a jelenlegi rendszer bővítésére.

5.1. Jövőbeli bővítési pontok

A szakdolgozat során sikerült kialakítani egy működő rendszert, amelyben a Tello drón ROS 2 környezetből vezérelhető, a kamera kép valós időben feldolgozódik, majd a mozgás a szimulációs platformon is megtörténik. Bár a rendszer a tervezett funkciók nagy részét megvalósítja, véleményem szerint még bőven van potenciál a jelenlegi rendszer bővítésére. A nyilvánvaló vonal az automatikus mozgás tovább gondolása. Jelenleg csak ellenőrzött körülmények között működik. Elsősorban a képfeldolgozás finomításával, a körök felismerése jóval kifinomultabb lehetne, ami óriása előrelépés lehetne. Ezenfelül a mozgás finomításában főleg azt a részt lehetne bővíteni vagy akár jobban megcsinálni.

A képfeldolgozásnál elsősorban azt a szimulációban ritkán fennálló esetet kellene lekezelni, amikor a kamera nagyon éles szögben látja a fánkokat. Ekkor a hough detektor sok kört fog találni, a kontúr detektor pedig csak elveszti a kört.

A mozgáson mindenképpen az szorul fejlesztésre - jelenleg ilyen brute force-os megoldás van -, hogy amikor a drón a fánk közelébe ér, akkor hogyan viselkedjen. Ámbátor ez a pont az előző ponttal is szorosan összefügg.

A Docker-alapú környezet optimalizálása is fontos lépés lehet, különösen a GPU-támogatás, a képkésés csökkentése és a fejlesztői felület egyszerűsítése érdekében. Végül a rendszer könnyen bővíthető lenne több drón együttes kezelése irányába, amely lehetőséget adna akár kooperatív robotikai kísérletek elvégzésére is.

6. Összegzés

A szakdolgozat célja egy olyan robotikai rendszer megvalósítása volt, amely a DJI Tello drónt képes ROS 2 környezetben vezérelni, a kameraképet valós időben feldolgozni, valamint a színes fánkokat detektáló algoritmusokkal felismerni és azok alapján automata mozgást végezni. A téma választását a robotika, a mesterséges intelligencia és az automatizálás iránti érdeklődés, illetve a drónok modern haditechnikában és iparban betöltött szerepe adta.

A rendszer megvalósítása ROS 2-re, Pythonra és Gazebo szimulációra épült, Docker konténerrel biztosítva a kompatibilitást az archivált Foxy környezettel. A projekt során több, egymással együttműködő node készült: a drón mozgását végző vezérlő, egy kézi irányítást biztosító modul, a kamera képet feldolgozó kétféle detektor (Hough-transzformáció és kontúr-alapú színes körkeresés), valamint az autonóm logikai egység, amely PID-alapú megközelítéssel vezeti a drónt a célobjektumok irányába.

A kidolgozott rendszer (ellenőrzött környezetben!) működőképes és stabil. A drón képes kézi és automatikus vezérlésre, a kameraképet sikeresen feldolgozta, és a színes karikákat több módszerrel is fel tudta ismerni. A szimuláció és a valós drón mozgása összhangba került, a kördetektorok vizuális visszajelzést adtak, illetve a PID-alapú szabályzás segítségével a drón képes volt a középpont felé igazítani a repülését.

7. Végszó

Össességében a szakdolgozat elérte a célját. A bevezetésben megfogalmazott kérdésekre így már konkrét válasz született: sikerült egy olyan robotikai alkalmazást készíteni, amely egyszerre működik valós és szimulált környezetben, felismer objektumokat, és ezek alapján automatikus döntéseket hoz.